

Spark DAG & Task Scheduling Internals

Prepared by : Kaviraj T U

How Spark Actually Executes Your Code

✓ When you write a Spark job, it may look like a simple sequence of transformations:

```
df.filter(...).groupBy(...).join(...)
```

- ✓ But Spark doesn't execute this line by line.
- ✓ Instead, it builds a **Directed Acyclic Graph (DAG)** of operations and schedules them intelligently across the cluster.
- ✓ Understanding this internal process helps you diagnose why jobs create multiple stages, where shuffles happen, and why some tasks take longer than others.

What is a DAG in Spark?

- ✓ A **DAG (Directed Acyclic Graph)** represents the logical execution flow of your Spark job.
- ✓ It consists of:
 - **Nodes** → Transformations
 - **Edges** → Data dependencies
- ✓ Key idea:
- ✓ Spark **doesn't execute transformations immediately**.
- ✓ It builds a DAG first, then optimizes and schedules execution.

✓ This is called **lazy evaluation**.

Spark Execution Flow (High Level)

✓ When you run a Spark job, the process typically follows this sequence:

- 1 **User Code** → Transformations defined
- 2 **Logical Plan** → Spark builds DAG
- 3 **DAG Scheduler** → Splits DAG into stages
- 4 **Task Scheduler** → Assigns tasks to executors
- 5 **Executors** → Run tasks on partitions

✓ Each layer plays a different role in distributed execution.

Narrow vs Wide Transformations

✓ The DAG is influenced heavily by the type of transformation.

👉 Narrow Transformations

✓ Examples:

- `map`
- `filter`
- `select`

✓ Characteristics:

- Each partition depends on only one parent partition
- No data movement across nodes

- Can be pipelined within the same stage

✓ Result:

- Faster execution
 - No shuffle required
-

👉 Wide Transformations

✓ Examples:

- `groupBy`
- `reduceByKey`
- `join`
- `distinct`

✓ Characteristics:

- Data must move between partitions
- Requires **shuffle**

✓ Result:

- Spark splits the job into **multiple stages**

✓ This is where performance costs increase.

Stage Creation in Spark

✓ Spark creates a new stage **whenever a shuffle occurs**.

✓ Example workflow:

```
Read Data
  ↓
Filter
```

↓
Join ← Shuffle happens here
↓
Aggregation

✓ Stages might look like:

- **Stage 1:** Read + Filter
- **Stage 2:** Shuffle + Join
- **Stage 3:** Aggregation

✓ Each stage consists of multiple **tasks**, executed in parallel.

What is a Task?

✓ A **task** is the smallest unit of work in Spark.

✓ Each task processes:

👉 **One partition of data**

✓ Example:

- Dataset has 200 partitions
- Stage will run **200 tasks**

✓ Tasks run on executors across the cluster.

What the DAG Scheduler Does

✓ The **DAG Scheduler**:

- Analyzes the DAG
- Identifies shuffle boundaries
- Divides job into stages

- Determines dependencies between stages

✓ It focuses on **stage-level scheduling**.

What the Task Scheduler Does

✓ The **Task Scheduler**:

- Receives stages from the DAG scheduler
- Assigns tasks to executors
- Handles retries if tasks fail
- Balances tasks across cluster nodes

✓ It operates at **task-level granularity**.

How to Inspect DAG in Spark UI

✓ In Spark UI:

✓ Navigate to:

Jobs → DAG Visualization

✓ You'll see:

- Stages
- Dependencies
- Shuffle boundaries

✓ This visualization shows how Spark interprets your code.

Common DAG Issues in Production

✓ Too Many Stages

Caused by repeated wide transformations.

✓ Large Shuffle Boundaries

Join or aggregation across huge datasets.

✓ Too Many Small Tasks

Due to excessive partitions.

✓ Skewed Tasks

One partition holding much more data.

Understanding DAG structure helps you diagnose these quickly.

Real Production Insight

✓ Many developers focus only on writing transformations.

✓ But performance tuning begins with understanding:

- Where stages split
- How tasks are distributed
- Where shuffles occur

✓ Spark execution is not just code — it's a distributed workflow graph.

Final Takeaway

✓ Spark does not execute operations sequentially.

✓ It builds a **DAG**, divides it into **stages**, and runs **tasks in parallel across partitions**.

✓ The moment a shuffle appears, Spark creates a new stage.

✓ If you can read a DAG visualization and immediately understand the execution flow, you've crossed an important milestone as a data engineer.

Thank you.!

Happy Learning...